

Soluzione ASSEMBLY (Traccia A):**MAIN**

```
%include 'utils.nasm'

section .data
    V dw 5, -3, 8, 4, 0, -18, 14, 5
    n equ ($-V)/2

section .bss
    i resd 1

section .text
    extern proc
    global _start
_start:

    ; CARICAMENTO PARAMETRI
    push i
    push n
    push V

    ; CHIAMATA A FUNZIONE
    call proc

    ; STAMPA RISULTATO
    printd dword [i]

    ; USCITA
esci:
    exit 0
```

PROCEDURA

```
section .data
    V equ 8
    n equ 12
    i equ 16

section .text
    global proc
proc:
    push EBP
    mov EBP, ESP
    pushad

    ; LETTURA PARAMETRI
    mov ESI, [EBP + n]
    mov EBX, [EBP + V]

    ; SOMMA ELEMENTI DI V
    xor AX, AX ; valore della somma
    xor EDI, EDI
ciclo_somma:
    cmp EDI, ESI
    jge avanti
    add AX, [EBX + EDI*2]
    inc EDI
    jmp ciclo_somma

    ; RICERCA INDICE
avanti:
    shl AX, 1
    mov CX, 3
    cwd
    div CX ; in AX ho i 2/3 della somma
    cmp DX, 0 ; se non è multiplo di 3
    je non_trovato ; non può esistere
    xor CX, CX ; valore della somma
    xor EDI, EDI
ciclo:
    cmp EDI, ESI
    jge non_trovato
    add CX, [EBX + EDI * 2]
    cmp CX, AX
    je trovato
    inc EDI
    jmp ciclo
non_trovato:
    mov EDI, -1
trovato:
    mov ECX, [EBP + i]
    mov [ECX], EDI
esci:
    popad
    pop EBP
    ret 12
```

Soluzione ASSEMBLY (Traccia B):**MAIN**

```
%include 'utils.nasm'

section .data
    V dw 4, -2, 3, 8, 0, 7, -15, 10
    n equ ($-V)/2

section .bss
    i resd 1

section .text
    extern proc
    global _start
_start:

    ; CARICAMENTO PARAMETRI
    push i
    push n
    push V

    ; CHIAMATA A FUNZIONE
    call proc

    ; STAMPA RISULTATO
    printd dword [i]

    ; USCITA
esci:
    exit 0
```

PROCEDURA

```
section .data
    V equ 8
    n equ 12
    i equ 16

section .text
    global proc
proc:
    push EBP
    mov EBP, ESP
    pushad

    ; LETTURA PARAMETRI
    mov ESI, [EBP + n]
    mov EBX, [EBP + V]

    ; SOMMA ELEMENTI DI V
    xor AX, AX ; valore della somma
    xor EDI, EDI
ciclo_somma:
    cmp EDI, ESI
    jge avanti
    add AX, [EBX + EDI*2]
    inc EDI
    jmp ciclo_somma

    ; RICERCA INDICE
avanti:
    mov CX, 3
    cwd
    div CX ; in AX ho 1/3 della somma
    cmp DX, 0 ; se non è multiplo di 3
    je non_trovato ; non può esistere
    xor CX, CX ; valore della somma
    xor EDI, EDI
ciclo:
    cmp EDI, ESI
    jge non_trovato
    add CX, [EBX + EDI * 2]
    cmp CX, AX
    je trovato
    inc EDI
    jmp ciclo
non_trovato:
    mov EDI, -1
trovato:
    mov ECX, [EBP + i]
    mov [ECX], EDI
esci:
    popad
    pop EBP
    ret 12
```